Sarah Benavides

CS 362 Lab

2/11/2026

Understanding and Analysis of Image Files

Part 1: Non-technical summary

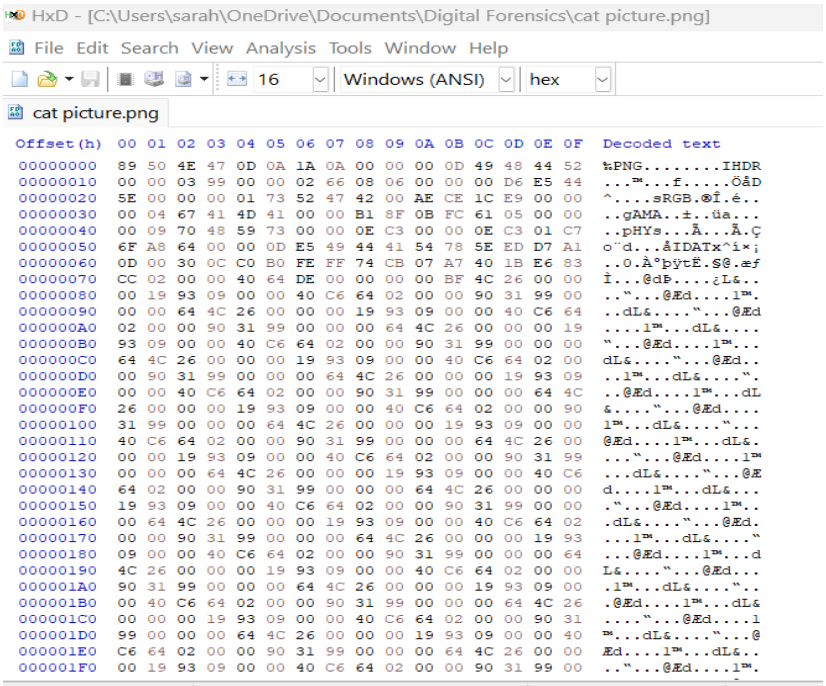
In this lab, we experimented with analyzing two different image files, .jpg and .png, in order to decipher whether they had been altered, and we also computed hash values for those image files, as well. Hash values are very important in the digital forensics world. A hash value is a way to identify a piece of data. There are multiple ways to create hash values for specific files: inputting certain commands into the Windows Command Prompt, using the HashCalc software, and writing specific lines of code in Python. During this lab, we also determined that small changes can be made to an image, like inputting a secret message within the image file, without affecting how it looks visually.

In addition to learning how hidden messages work within a file, this lab also taught me how even minor adjustments to a file can result in completely different hash values. This confirmed how sensitive hash values are and why they are used to verify the integrity of a file. When we compared the original and modified image files, it was shown that just visualizing it on its own will not reveal all the changes that took place.

Part 2: Technical summary

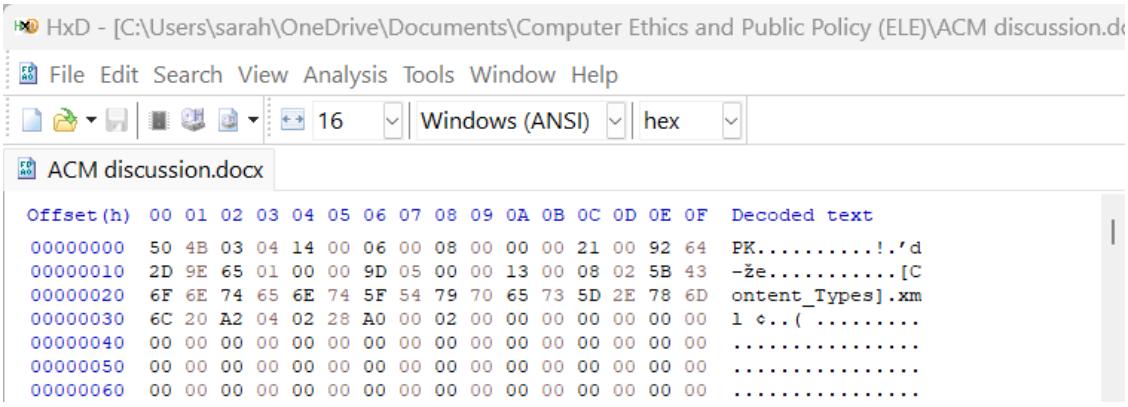
To start this lab, I visited the website pixabay.com/ to download a free .jpg image. After saving the image successfully, I opened the Microsoft Paint app and saved the .jpg as a .png file. This is the image I chose.

I then downloaded the HxD software. This is where I opened my newly saved .png file and upon selecting “open”, the screen filled with hexadecimal characters. The hexadecimal signature for the .png was **89 50 4E 47 0D 0A 1A 0A.**

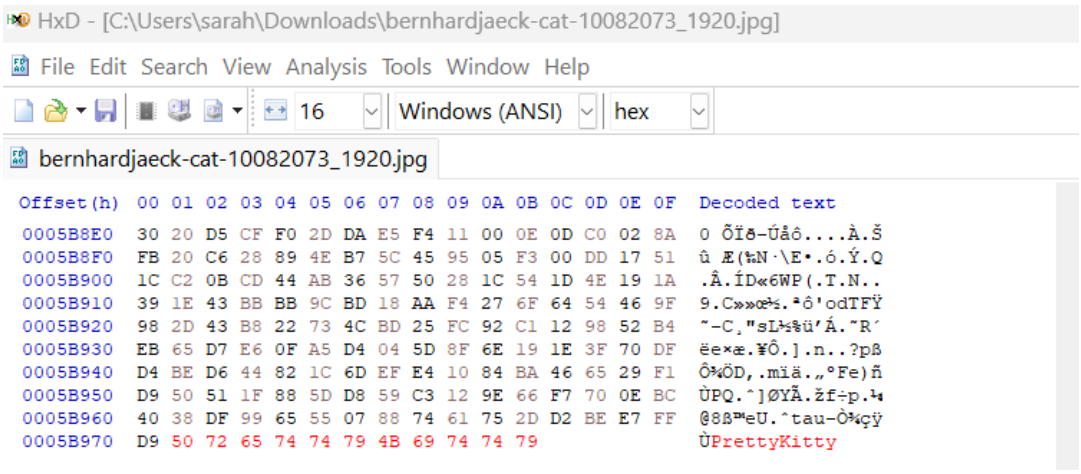


I then opened the saved .jpg file within the HxD software, and the hexadecimal signature for it was **FF D8 FF E0 00 10 4A 46 49 46 00 01**.

In the next step, I chose a few random files from my computer and verified their file signature using the HxD software. The files I chose were **docx** and **pptx**. The docx file and the pptx file had the same hexadecimal signature, which was **50 4B 03 04**.



For the third step of part one, I took a .jpg file and opened it in the HxD software. I chose to use the same cat picture I picked at the beginning of the lab to make it easier to compare. After opening the .jpg in the software, I scrolled to the bottom of the file, typed “PrettyKitty”, saved the file, and exited the HxD software.

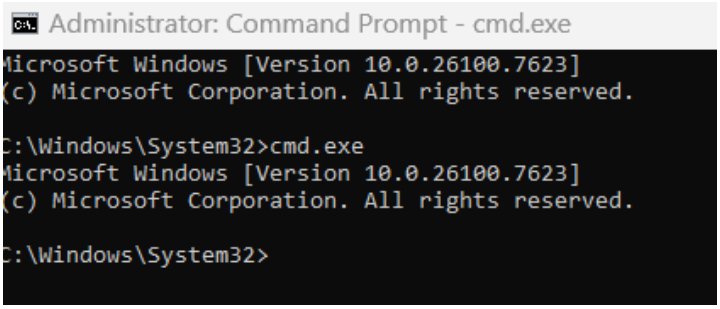


I then compared the original image with the secret hidden message image, and both files were the same size. I also opened both images, and the secret hidden message image had no noticeable visual differences. This is because the secret message was placed at the very end of the file, and this simply adds more data to the file instead of creating massive changes to the picture.

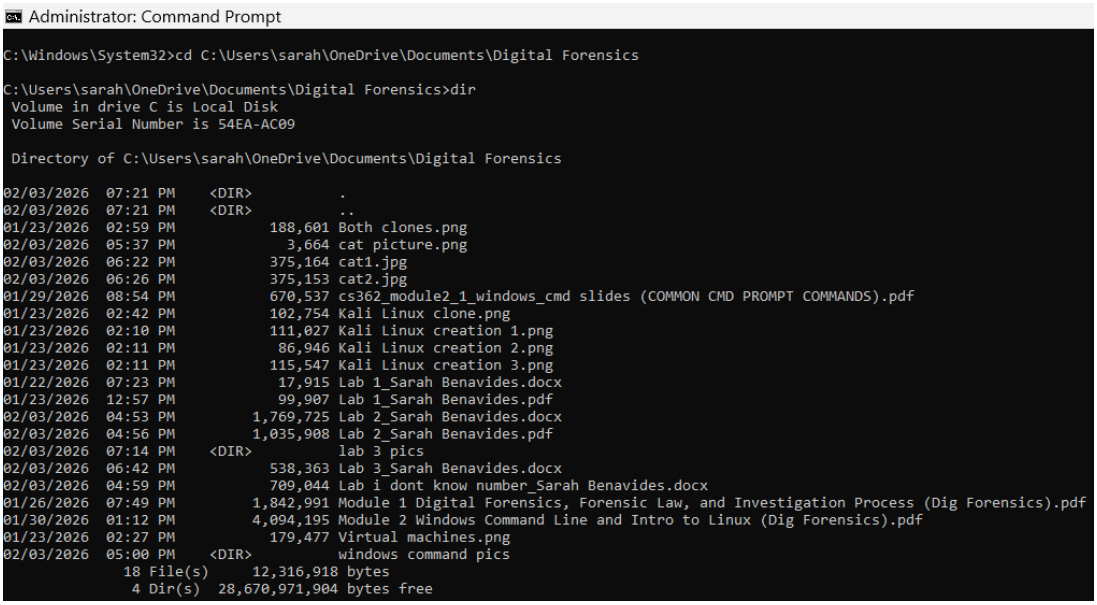
Original:

Secret Hidden Message:

For part two, I began by opening a command prompt window/session and running the file **cmd.exe** as administrator. This command lists the version of Windows running on my computer as well as the copyright note.

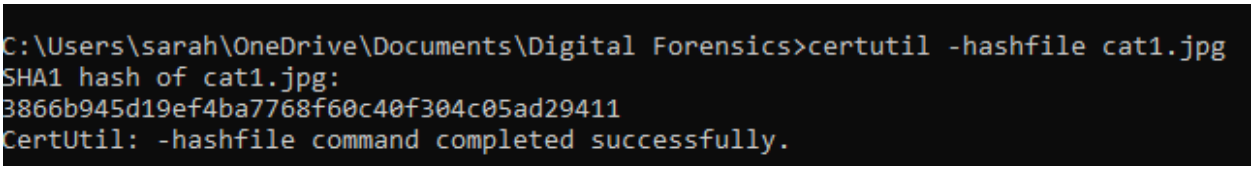


After running the file, I changed the working directory to the directory where the two .jpg images are stored. This directory is called **C:\Users\sarah\OneDrive\Documents\Digital Forensics**.

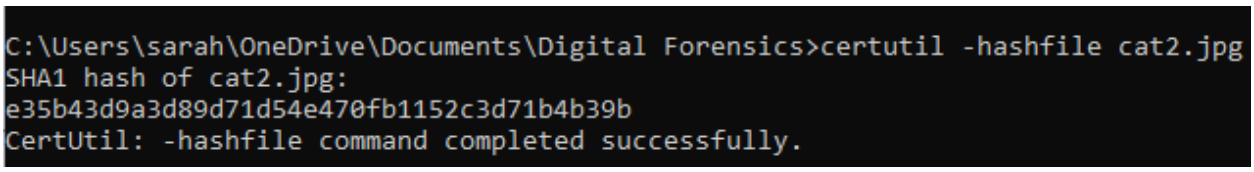


By using the **certutil** command in the command prompt, I was able to compute the hash values of the non-modified and modified .jpg files used in the first part of the lab.

Non-modified .jpg:



Modified .jpg:



The hash values for the non-modified and the modified .jpg are completely different from one another because they are two completely different and separate files.

When I correctly wrote the following command of

FORFILES /M *.jpg /C “cmd /c echo @FILE”,

the output included both the non-modified and modified .jpg files.

```
C:\Users\sarah\OneDrive\Documents\Digital Forensics>FORFILES /M *.jpg /C "cmd /c echo @FILE"

"cat1.jpg"
"cat2.jpg"
```

After successfully inputting the command

FORFILES /M *.jpg /C “cmd /c certutil -hashfile @FILE”

into the prompt, the output listed the hash algorithm plus the hash values of cat1.jpg and cat2.jpg.

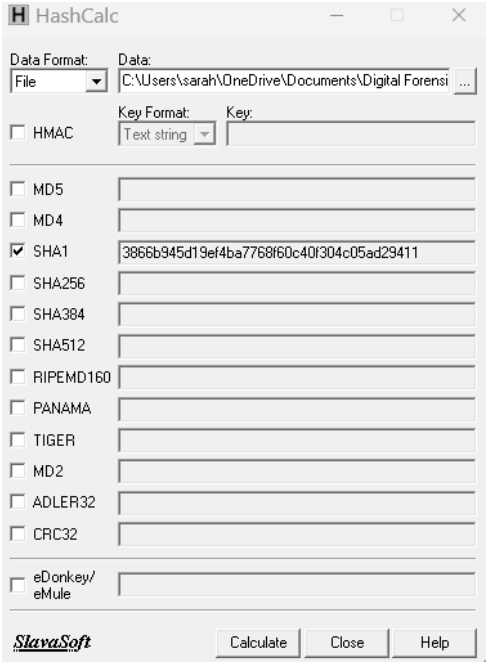
```
C:\Users\sarah\OneDrive\Documents\Digital Forensics>FORFILES /M *.jpg /C "cmd /c certutil -hashfile @FILE"

SHA1 hash of cat1.jpg:
3866b945d19ef4ba7768f60c40f304c05ad29411
CertUtil: -hashfile command completed successfully.
SHA1 hash of cat2.jpg:
e35b43d9a3d89d71d54e470fb1152c3d71b4b39b
CertUtil: -hashfile command completed successfully.
```

In order to compute hash values of all or part of the files in the working directory in a single command, the user would remove the “.jpg” from the command above. The following is how that command would appear:

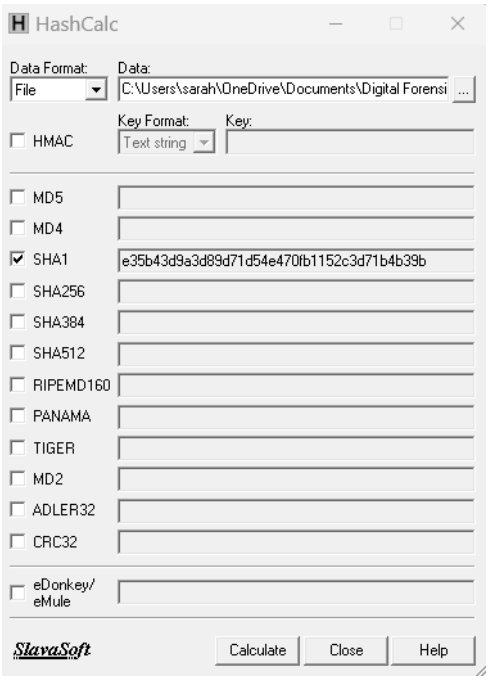
```
C:\Users\sarah\OneDrive\Documents\Digital Forensics>FORFILES /M * /C "cmd /c certutil -hashfile @FILE"

SHA1 hash of Both clones.png:
a42a7bd714340f3802015596f3281df9a80da4a0
CertUtil: -hashfile command completed successfully.
SHA1 hash of cat picture.png:
e3d4525f0aed6b6e04014f85e79ccce87bf45b9c
CertUtil: -hashfile command completed successfully.
SHA1 hash of cat1.jpg:
3866b945d19ef4ba7768f60c40f304c05ad29411
CertUtil: -hashfile command completed successfully.
SHA1 hash of cat2.jpg:
e35b43d9a3d89d71d54e470fb1152c3d71b4b39b
CertUtil: -hashfile command completed successfully.
SHA1 hash of cs362_module2_1_windows_cmd slides (COMMON CMD PROMPT COMMANDS).pdf:
6d9a36d9ac3de35bf5b367456c7cb0224ad4872c
CertUtil: -hashfile command completed successfully.
SHA1 hash of Kali Linux clone.png:
40e689e4c60060f70d49a135aafc1872fe05c006
CertUtil: -hashfile command completed successfully.
SHA1 hash of Kali Linux creation 1.png:
c43e752731e209ed29e36b7f916b61fc412a2a92
CertUtil: -hashfile command completed successfully.
SHA1 hash of Kali Linux creation 2.png:
07cb3a4180728fd388e6e2a28c2751fd954b9281
CertUtil: -hashfile command completed successfully.
SHA1 hash of Kali Linux creation 3.png:
8462798f528c4f61ecb3546f2a6f56f4e2e5edde
CertUtil: -hashfile command completed successfully.
SHA1 hash of Lab 1_Sarah Benavides.docx:
8b1816a4bc46d99ed26f86a58be618f62701e5cd
CertUtil: -hashfile command completed successfully.
SHA1 hash of Lab 1_Sarah Benavides.pdf:
5b2c91e46c3cc3b5d2e1cc39682034b214c4a682
CertUtil: -hashfile command completed successfully.
SHA1 hash of Lab 2_Sarah Benavides.docx:
84bd6c009ea2393014e6a1cc593a86983eeb74d7
CertUtil: -hashfile command completed successfully.
SHA1 hash of Lab 2_Sarah Benavides.pdf:
52fc7e2d0e705b1681d4435fa8529d73379523f6
CertUtil: -hashfile command completed successfully.
CertUtil: -hashfile command FAILED: 0x80070002 (WIN32: 2 ERROR_FILE_NOT_FOUND)
CertUtil: The system cannot find the file specified.
CertUtil: -hashfile command FAILED: 0x80070020 (WIN32: 32 ERROR_SHARING_VIOLATION)
CertUtil: The process cannot access the file because it is being used by another process.
SHA1 hash of Lab i dont know number_Sarah Benavides.docx:
9a88cb5c9c10d7177d569470bb9b782a753b24cc
CertUtil: -hashfile command completed successfully.
SHA1 hash of Module 1 Digital Forensics, Forensic Law, and Investigation Process (Dig Forensics).pdf:
b50fdad5281d9853bc0a38e65a82945a21c488c6
CertUtil: -hashfile command completed successfully.
SHA1 hash of Module 2 Windows Command Line and Intro to Linux (Dig Forensics).pdf:
f6469abf032ce52557f19b57b902ae2b1a55e963
CertUtil: -hashfile command completed successfully.
SHA1 hash of Virtual machines.png:
b465785c1a50739f63a6384a6c28b48cbb75871d
CertUtil: -hashfile command completed successfully.
CertUtil: -hashfile command FAILED: 0x80070002 (WIN32: 2 ERROR_FILE_NOT_FOUND)
```

In the third part of this lab, we used the HashCalc software. I located the two saved image files that were used previously and selected them one at a time. I selected the SHA1 hash function name because that is the one that was used for both image files. After calculating each hash, the results were as follows:

Non-modified image file:



Modified image file:



The results from using the HashCalc software and the certutil command in steps 14 and 15 are exactly the same.

To complete this part of the lab, the latest version of PowerShell was installed by entering the command **winget search --id Microsoft.PowerShell** into the command prompt.

```
C:\Users\sarah\OneDrive\Documents\Digital Forensics>winget search --id Microsoft.Powershell
The `msstore` source requires that you view the following agreements before using.
Terms of Transaction: https://aka.ms/microsoft-store-terms-of-transaction
The source requires the current machine's 2-letter geographic region to be sent to the backend service
to function properly (ex. "US").

Do you agree to all the source agreements terms?
[Y] Yes  [N] No: Y
Name               Id                           Version Source
-----------------------------------------------------------------
PowerShell         Microsoft.PowerShell         7.5.4.0 winget
PowerShell Preview Microsoft.PowerShell.Preview 7.6.0.6 winget
```

After installation was complete, I opened Windows PowerShell and ran it as administrator. The first command I entered was **SET-LOCATION -PATH "C:\Users\sarah\OneDrive\Documents\Digital Forensics"** to change the directory. By using the GET-FILEHASH command, we are able to compute the digests of both the non-modified and modified .jpg files.

```
PS C:\Users\sarah\OneDrive\Documents\Digital Forensics> GET-FILEHASH -Algorithm SHA1
*.jpg

Algorithm       Hash
---------       ----
SHA1            3866B945D19EF4BA7768F60C40F304C05AD29411
SHA1            E35B43D9A3D89D71D54E470FB1152C3D71B4B39B
```

I already had Anaconda downloaded for a previous class, but I was unable to find out if it was Anaconda3 or not. After downloading Anaconda3, I was able to open the Anaconda Prompt and run it as administrator. The first task I completed was updating the Anaconda packages. For some reason, when inputting **conda update anaconda** I received an error that the package was not installed. I tried changing the command to **conda update conda,** and everything ran smoothly.

```
(base) C:\Windows\System32>conda update conda
3 channel Terms of Service accepted
Retrieving notices: done
Channels:
 - defaults
Platform: win-64
Collecting package metadata (repodata.json): done
Solving environment: done

## Package Plan ##

  environment location: C:\Users\sarah\anaconda3\anaconda3

  added / updated specs:
    - conda


The following packages will be downloaded:

    package                    |            build
    ---------------------------|-----------------
    ca-certificates-2025.12.2  |       haa95532_0         125 KB
    certifi-2026.01.04         |  py313haa95532_0         148 KB
    conda-26.1.0               |  py313haa95532_0         1.2 MB
    openssl-3.0.19             |       hbb43b14_0         6.9 MB
    ------------------------------------------------------------
                                           Total:         8.4 MB

The following packages will be UPDATED:

  ca-certificates                     2025.11.4-haa95532_0 --> 2025.12.2-haa95532_0
  certifi                       2025.11.12-py313haa95532_0 --> 2026.01.04-py313haa95532_0
  conda                            25.11.0-py313haa95532_0 --> 26.1.0-py313haa95532_0
  openssl                             3.0.18-h543e019_0 --> 3.0.19-hbb43b14_0


Proceed ([y]/n)? y


Downloading and Extracting Packages:

Preparing transaction: done
Verifying transaction: done
Executing transaction: done
```

The next command I input was **conda install spyder** to install the last version of Spyder on my system.

```
(base) C:\Windows\System32>conda install spyder
Channels:
 - defaults
Platform: win-64
Collecting package metadata (repodata.json): done
Solving environment: done

## Package Plan ##

  environment location: C:\Users\sarah\anaconda3

  added / updated specs:
    - spyder


The following packages will be downloaded:

    package                    |            build
    ---------------------------|-----------------
    aiohttp-3.11.10            |  py312h827c3e9_0         900 KB
    bcrypt-5.0.0               |  py312h114bc41_0         147 KB
    ca-certificates-2025.12.2  |       haa95532_0         125 KB
    cattrs-24.1.2              |  py312haa95532_0         137 KB
    certifi-2026.01.04         |  py312haa95532_0         148 KB
    ipython_pygments_lexers-1.1.1|  py312haa95532_0          19 KB
    lsprotocol-2025.0.0        |  py312haa95532_0         236 KB
    openssl-3.0.19             |       hbb43b14_0         6.9 MB
    propcache-0.4.1            |  py312h02ab6af_0          58 KB
    python-lsp-ruff-2.3.0      |  py312haa95532_0          32 KB
    python-lsp-server-1.13.1   |  py312hbc747e5_0         206 KB
    qtawesome-1.4.0            |  py312haa95532_0         1.9 MB
    qtconsole-5.7.0            |  py312haa95532_0         278 KB
    ruff-0.14.10               |  py312h114bc41_0         9.9 MB
    spyder-6.1.0               |  py312h2ba046a_2        10.7 MB
    spyder-kernels-3.1.2       |  py312h4442805_0         374 KB
    ucrt-10.0.22621.0          |       haa95532_0         620 KB
    vc14_runtime-14.44.35208   |      h4927774_10         825 KB
    vs2015_runtime-14.44.35208 |      ha6b5a95_10          19 KB
    yarl-1.22.0                |  py312h02ab6af_0         144 KB
    ------------------------------------------------------------
                                           Total:        33.6 MB

The following NEW packages will be INSTALLED:

  cattrs             pkgs/main/win-64::cattrs-24.1.2-py312haa95532_0
  ipython_pygments_~ pkgs/main/win-64::ipython_pygments_lexers-1.1.1-py312haa95532_0
  lsprotocol         pkgs/main/win-64::lsprotocol-2025.0.0-py312haa95532_0
  propcache          pkgs/main/win-64::propcache-0.4.1-py312h02ab6af_0
  python-lsp-ruff    pkgs/main/win-64::python-lsp-ruff-2.3.0-py312haa95532_0
  ruff               pkgs/main/win-64::ruff-0.14.10-py312h114bc41_0
  ucrt               pkgs/main/win-64::ucrt-10.0.22621.0-haa95532_0
  vc14_runtime       pkgs/main/win-64::vc14_runtime-14.44.35208-h4927774_10

The following packages will be UPDATED:

  aiohttp                              3.10.5-py312h827c3e9_0 --> 3.11.10-py312h827c3e9_0
  bcrypt                               3.2.0-py312h2bbff1b_1 --> 5.0.0-py312h114bc41_0
  ca-certificates                      2024.12.31-haa95532_0 --> 2025.12.2-haa95532_0
```

```
  certifi                         2024.12.14-py312haa95532_0 --> 2026.01.04-py312haa95532_0
  openssl                             3.0.15-h827c3e9_0 --> 3.0.19-hbb43b14_0
  python-lsp-server                1.12.0-py312hfc267ef_0 --> 1.13.1-py312hbc747e5_0
  qtawesome                         1.3.1-py312haa95532_0 --> 1.4.0-py312haa95532_0
  qtconsole                         5.6.1-py312haa95532_0 --> 5.7.0-py312haa95532_0
  spyder                            6.0.1-py312haa95532_0 --> 6.1.0-py312h2ba046a_2
  spyder-kernels                    3.0.0-py312hfc267ef_0 --> 3.1.2-py312h4442805_0
  vs2015_runtime                   14.40.33807-h98bb1dd_1 --> 14.44.35208-ha6b5a95_10
  yarl                             1.11.0-py312h827c3e9_0 --> 1.22.0-py312h02ab6af_0


Proceed ([y]/n)? y


Downloading and Extracting Packages:

Preparing transaction: done
Verifying transaction: done
Executing transaction: / Terminal profiles are not available for system level installs

Terminal profiles are not available for system level installs

| C:\Users\sarah\anaconda3\Lib\site-packages\menuinst\platforms\win.py:70: UserWarning: Quick launch menus are not avais
  warnings.warn("Quick launch menus are not available for system level installs")
/ Terminal profiles are not available for system level installs

done
```

Once Spyder was successfully updated, I was able to run the Spyder program and create a new .py file and name it lab3_part3. I followed the Python code that was provided in this lab document and added my specific .jpg file names: “cat1.jpg” and “cat2.jpg”. After running this code, the hash values for each .jpg file were created.

```python
import hashlib

f1 = open("cat1.jpg", 'rb')
f2 = open("cat2.jpg", 'rb')

#get files' data
data1 = f1.read()
data2 = f2.read()

#compute SHA1 hash values (digests) and print them in hexidecimal format
print(hashlib.sha1(data1).hexdigest())
print(hashlib.sha1(data2).hexdigest())
```

```
In [2]: %runfile 'C:/Users/sarah/OneDrive/
Documents/Digital Forensics/lab3_part3.py'
wdir
3866b945d19ef4ba7768f60c40f304c05ad29411
e35b43d9a3d89d71d54e470fb1152c3d71b4b39b
```

The following code should not be used by digital forensics investigators because the "Image.open" part of the code strictly opens the part that is visible from PIL. Cat1.jpg and cat2.jpg have completely different data outside the visual part, which is why the hash values differ in the code that was run in the previous step. After running the code, the hash values turn out to be the same for each .jpg image, even though they are different.

```python
import hashlib
from PIL import Image

f1 = Image.open("cat1.jpg").tobytes()
f2 = Image.open("cat2.jpg").tobytes()


#compute SHA1 hash values (digests) and print them in hexidecimal
print(hashlib.sha1(f1).hexdigest())
print(hashlib.sha1(f2).hexdigest())
```

```
In [3]: %runfile 'C:/Users/
sarah/OneDrive/Documents/
Digital Forensics/
lab3_CodeNotToUse.py' --wdir
ee81bb8a7cf846eec7dc839fc81c32
f0ad1a20e4
ee81bb8a7cf846eec7dc839fc81c32
f0ad1a20e4
```

In the second-to-last part of the lab, our goal is to hide a secret in an image without exposing that secret. I created a new Python file, named it lab_part6, and added the following code to it.

```python
from PIL import ImageChops
from numpy import asarray
import hashlib
from PIL import Image

#create handler to image dog.png
image = Image.open("dog.png")

#store the image cat.png using the PIL.Image save
#method to unify the way in which images are saved
#in this lab. pydog.png is referred to as the
#original image

image.save("dog1.png")

#get width and height of image
w, h = image.size

#create secret, load the image to be able to
#modify the values of its pixels, and embed it
#in the uploaded image
secret = b'ATTACK9:40AM'
im = image.load()
im[30, 230] = (secret[0], secret[1], secret[2])
im[100, 400] = (secret[3], secret[4], secret[5])
im[400, 100] = (secret[6], secret[7], secret[8])
im[200, 280] = (secret[9], secret[10], secret[11])

#save the modified image and close the original
image.save("dog2.png")
image.close()
```

Name	Type	Size	Value
h	int	1	1920
im	PixelAccess	1	PixelAccess object
image	Image	[1200, 1920]	<PngImageFile @ 0x1F5C6E73790> Mode: RGBA
secret	bytes	12	ATTACK9:40AM
w	int	1	1200

I ran the code, and the files dog1.png and dog2.png were created along with the chart above listing multiple variables that were asked about in the code. I compared dog.png to both dog1 and dog2, and I did not see any visual differences in the images. In the next snippet of code, we are trying to find the differences between the two dog PNG images using Python. The output lists the locations where we stored the secret, as well as the hash values in hexadecimal format.

```python
from PIL import ImageChops
from numpy import asarray
import hashlib
from PIL import Image

#open the two images
dog_original = Image.open("dog1.png")
dogsecret = Image.open("dog2.png")

#get width and height of the image
w, h = dog_original.size

#find the difference between the two images
difference = ImageChops.difference(dog_original, dogsecret)

#convert different image into a numpy array to be
#able to perform mathematical operations on it
data = asarray(difference)

#display indices of pixels' with different != 0
for r in range(h):
    for c in range(w):
        if sum(data[r, c]):
            print(r, c, sep='\t')

#open files for reading (in binary)
f1 = open("dog1.png", 'rb')
f2 = open("dog2.png", 'rb')

data1 = f1.read()
data2 = f2.read()
```

```python
#compute and display sha256 sigest for two images
sha256f1 = hashlib.sha256()
sha256f1.update(data1)
sha256f2 = hashlib.sha256()
sha256f1.update(data2)
print(sha256f1.hexdigest(), sha256f2.hexdigest(), sep='\n')

#compute and display sha1 digests of the two images
print("SHA1 Digests: ")
print(hashlib.sha1(data1).hexdigest())
print(hashlib.sha1(data2).hexdigest())
```

```
In [9]: %runfile 'C:/Users/sarah/OneDrive/Documents/Digital Forensics/lab3_part6_2.py'
--wdir
c:\users\sarah\onedrive\documents\digital forensics\lab3_part6_2.py:30:
RuntimeWarning: overflow encountered in scalar add
  if sum(data[r, c]):
100 400
230 30
280 200
400 100
a8f1dd00e56205c6bab2ff90444b1585122fc59d86a0e82c4933a6962806cc10
e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855
SHA1 Digests:
2f51e17aa8a6823b95cb55e23e81ac6552212b0a
8e33c06b6d1129ffae5fb9d405e943cb0ee437e1
```

The last part of this lab focuses on Fuzzy Hashing. Fuzzy hashing is used to compare files. In this final step, we compare the original PNG file with the secret PNG file. After I input the following code,

```
import ppdeep

#compute the hash values from files
h1 = ppdeep.hash_from_file("dog1.png")
h2 = ppdeep.hash_from_file("dog2.png")

print(h1, h2, sep='\n')

print('\nLevel of similarity: ')
print(ppdeep.compare(h1, h2))
```

the output showed that the level of similarity is zero. Each hash value was also completely different from each other.

```
In [1]: %runfile 'C:/Users/sarah/OneDrive/Documents/Digital
Forensics/lab3_part7.py' --wdir
98304:3EoRh0YiK3xRlL8JTa4jLHuxcTuOzkWuh4hfyrpX:
3E42ubR8Y4jLOxOuwseIrZ
49152:3A9rEDL/u7ZBGc8QD+YVzrBHzojU/
krIiiSwAbODao00MQ5NFnp1CJjCs5ASzwvX:38/Lqy0Q/vF4A/00dvFw5pzm/
b1ln

Level of similarity:
0
```

Part 3: References

1. What is a Hash Value?
2. Will a Secret Hidden Message Added to a JPG File Make it Distorted?
3. PIL Information
4. Module 3: Handling Image Files Lecture Recordings Parts 1 and 2
5. Lab 3 Handling Imaging Files